Towards Dev · Follow publication

Low-Latency C++: A Practical Guide to Cache Warming

How to use prefetching, cache warming, and memory tricks to eliminate latency in hot paths

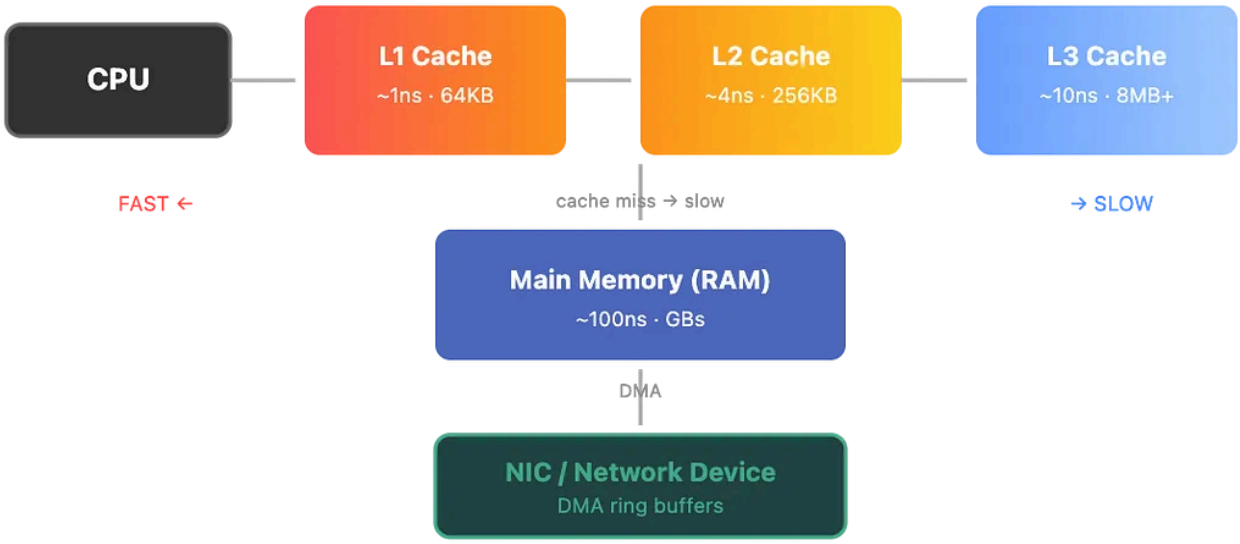
7 min read · Apr 14, 2026

Sagar Following

What Is Cache Warming?

Your CPU doesn't talk to RAM directly — well, it does, but it *hates* it. Fetching from main memory takes ~100 nanoseconds. Fetching from L1 cache? About 1 nanosecond. That's a 100x difference.

Cache warming is the practice of **pre-loading data into CPU cache before you actually need it**, so when the hot path runs, everything is already sitting right there.



Think of it like pre-heating an oven. You don't wait until the food is in to turn it on.

Prefetch Primitives: Your Main Tools

`__builtin_prefetch` (GCC/Clang)

This is the one you'll reach for first. It's a compiler built-in that tells the CPU: "hey, I'm going to need this memory soon."

```
// Signature:
// __builtin_prefetch(const void *addr, int rw, int locality)
//
//   rw:       0 = read (default), 1 = write
//   locality: 0 = no temporal locality (use once, evict soon)
//             1 = low temporal locality
//             2 = moderate temporal locality
//             3 = high temporal locality (keep in all cache levels)

void process_orders(struct Order *orders, int count) {
    for (int i = 0; i < count; i++) {
        // Prefetch 4 iterations ahead — gives the CPU time
        // to actually fetch it before we get there
        if (i + 4 < count) {
            __builtin_prefetch(&orders[i + 4], 0, 3);
        }
```

```
            execute_order(&orders[i]);
        }
    }
```

The “4 ahead” number isn’t magic — you tune it based on how much work `execute_order` does. More work per iteration = prefetch further ahead.

`_mm_prefetch` (Intel Intrinsics)

Same idea, different interface. This one comes from **`<xmmintrin.h>`** and maps directly to x86 prefetch instructions.

```
#include <xmmintrin.h>

// _MM_HINT_T0  → prefetch into L1, L2, L3  (like locality=3)
// _MM_HINT_T1  → prefetch into L2, L3      (like locality=2)
// _MM_HINT_T2  → prefetch into L3 only     (like locality=1)
// _MM_HINT_NTA → non-temporal, minimize cache pollution

void warm_buffer(const char *buf, size_t len) {
    for (size_t i = 0; i < len; i += 64) {  // 64 = cache line size
        _mm_prefetch(buf + i, _MM_HINT_T0);
    }
}
```

I’ll use **`__builtin_prefetch`** for the rest of this post since it's more portable, but they compile down to the same instructions on x86.

The `volatile` Trick for Dry Reads

Sometimes you need to actually *read* memory to pull it into cache, but the compiler is too smart — it sees that you never use the result and optimizes your read away. Classic.

```
void warm_data(const char *data, size_t len) {
    for (size_t i = 0; i < len; i += 64) {
        // Without volatile, the compiler deletes this entire loop.
        // It thinks: "you read it, did nothing with it... gone."
        volatile char sink = data[i];
        (void)sink;
    }
}
```

The `volatile` keyword forces the compiler to actually perform the read. It's ugly, but it works reliably.

Dry-Run Execution Mode

Here's where it gets interesting. Instead of prefetching individual memory addresses, what if you just... *run your entire hot path* once, but in a mode where it doesn't actually do anything?

```
struct Engine {
    int dry_run;  // 1 = warming up, 0 = live
    // ... other fields
};

void handle_packet(struct Engine *eng, struct Packet *pkt) {
    // This touches all the same cache lines as a real execution
    struct Order order;
    parse_packet(pkt, &order);       // warms parser structures
    int valid = validate(&order);     // warms validation tables

    if (eng->dry_run)
        return;  // bail before side effects

    send_to_exchange(&order);         // only in live mode
}
```

The beauty is you don't have to figure out *which* addresses to prefetch. The code itself warms exactly what it needs.

But there's a problem — that `if (eng->dry_run) return` introduces a branch. And on startup, the branch predictor has no history, so it might mispredict. We can do better.

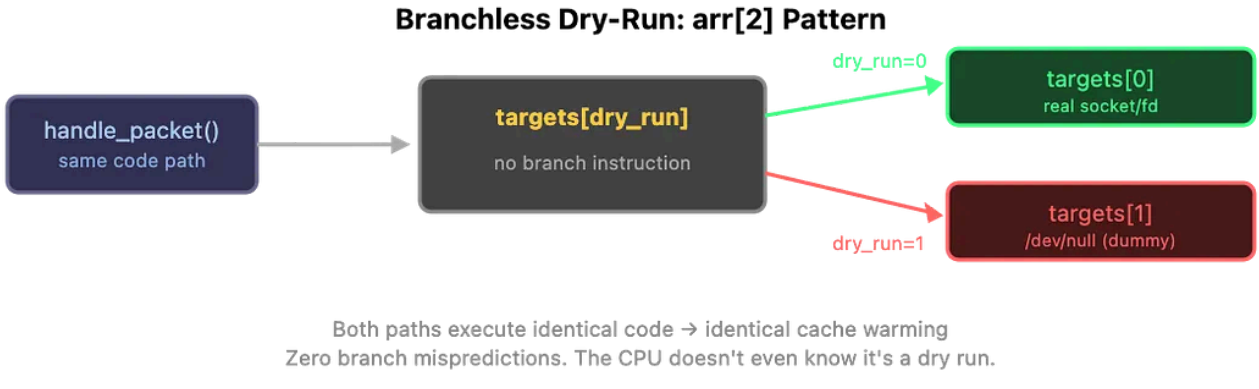
Branchless Dry-Run with `arr[2]`

This is my favorite trick. Instead of branching, you use a two-element array as a destination selector:

```
// Two targets: index 0 = real destination, index 1 = dummy (throwaway)
int send_targets[2];  // [0] = real fd/socket, [1] = /dev/null or dummy

void handle_packet(struct Engine *eng, struct Packet *pkt) {
    struct Order order;
    parse_packet(pkt, &order);
    validate(&order);

    // dry_run is 0 or 1 — no branch needed
    // Live mode:  writes to send_targets[0] (real)
    // Dry mode:   writes to send_targets[1] (throwaway)
    write(send_targets[eng->dry_run], &order, sizeof(order));
}
```



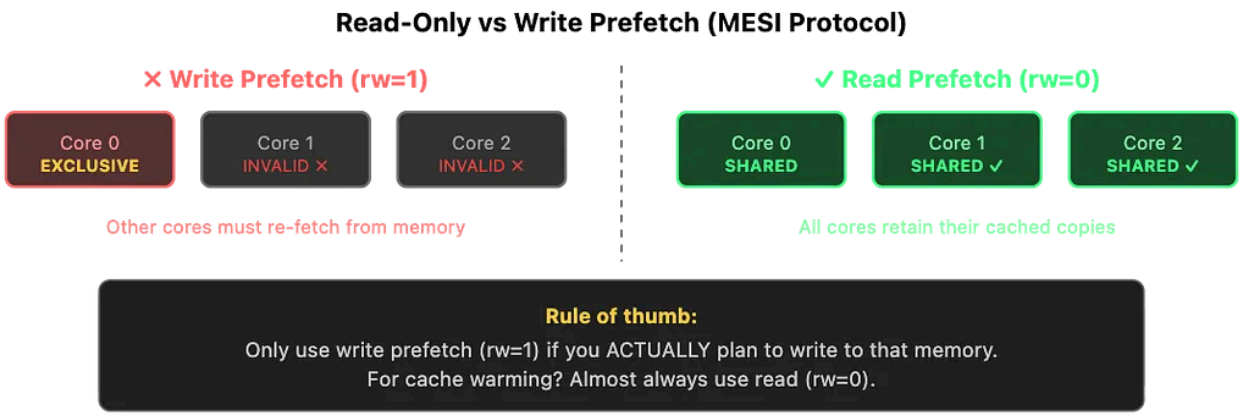
Zero branches. The CPU pipeline stays full. The cache gets warmed with the exact same access pattern as production. Chef's kiss.

Read-Only Warming (Play Nice with Other Threads)

This one bit me in production. If you prefetch with write intent, the CPU takes an exclusive lock on that cache line (MESI protocol — the line goes to "Modified" or "Exclusive" state). This **invalidates** the line on every other core.

```
// BAD — if other threads are reading this data, you just
// blew up their caches for no reason
__builtin_prefetch(&shared_data[i], 1, 3);  // rw=1 → write intent

// GOOD — read-only prefetch, cache line stays in "Shared" state
// all cores can hold a copy simultaneously
__builtin_prefetch(&shared_data[i], 0, 3);  // rw=0 → read intent
```



> ***Rule:*** *Unless you're* actually *going to write to the memory during the hot path, always warm with read-only prefetches.*

Deep Warming: All the Way into the NIC

In ultra-low-latency networking (think HFT or real-time telemetry), you don't just warm CPU caches. You also warm the path into the NIC.

NIC ring buffers live in DMA-accessible memory. When a packet arrives, the NIC writes it to a pre-allocated buffer via DMA — and that buffer might be cold. If you pre-touch those DMA buffers, the TLB entries and page table mappings are already resolved.

```
// Warm the NIC RX ring buffers
void warm_nic_rx_ring(struct ring_buffer *ring) {
    for (int i = 0; i < ring->size; i++) {
        volatile char sink = ring->buffers[i].data[0];
        (void)sink;

        // Also prefetch the descriptor — the NIC writes
        // status/length here via DMA
        __builtin_prefetch(&ring->descriptors[i], 0, 3);
    }
}
```

This ensures that when the first real packet hits, the kernel doesn't stall on a TLB miss or a page fault while trying to hand you the data.

Optimize: Respect the Cache Line

Here's a mistake I see all the time. People warm every byte:

```
// WASTEFUL — reading every byte, but cache loads in 64-byte chunks
for (size_t i = 0; i < len; i++) {
    volatile char sink = data[i];
}
```

Your CPU fetches memory in cache lines, typically 64 bytes. One touch per line is enough:

```
// CORRECT — one touch per cache line
// You can check your line size: getconf LEVEL1_DCACHE_LINESIZE
#define CACHE_LINE_SIZE 64

void warm_buffer(const char *data, size_t len) {
    for (size_t i = 0; i < len; i += CACHE_LINE_SIZE) {
        __builtin_prefetch(data + i, 0, 3);
    }
}
```

That's 64x fewer instructions for the same warming effect.

Cap It: Don't Warm More Than the Cache Can Hold

This is the one that most people miss entirely, and it can make your warming code actively *harmful*.

If you have 8MB of L3 cache and you try to warm 32MB of data, the first 24MB you warmed gets **evicted** before you ever use it. You wasted cycles and polluted the cache with the tail end of your data.

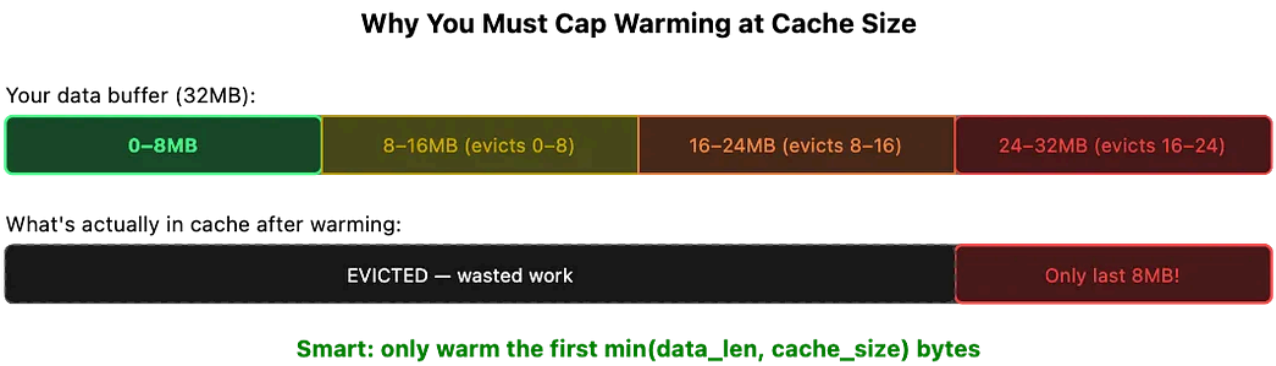
```
#include <unistd.h>

size_t get_l3_cache_size(void) {
    // Linux-specific; returns L3 size in bytes
    long size = sysconf(_SC_LEVEL3_CACHE_SIZE);
    return (size > 0) ? (size_t)size : 8 * 1024 * 1024;  // 8MB fallback
}

void warm_buffer_smart(const char *data, size_t len) {
    size_t max_warm = get_l3_cache_size();

    // Only warm up to what the cache can actually hold
    size_t warm_len = (len < max_warm) ? len : max_warm;

    for (size_t i = 0; i < warm_len; i += CACHE_LINE_SIZE) {
        __builtin_prefetch(data + i, 0, 3);
    }
}
```



If your hot data is larger than your cache, you need to prioritize *which* part you warm — usually the structures accessed first in your hot path.

Putting It All Together

Here's a realistic-ish warming routine for a packet processing engine:

```
#define CACHE_LINE 64

struct Engine {
    int dry_run;
    int output_fds[2];     // [0]=real socket, [1]=dummy fd (/dev/null)
    struct LookupTable *table;
    struct RingBuffer *rx_ring;
};

void warm_engine(struct Engine *eng) {
    size_t cache_cap = get_l3_cache_size();
    size_t warmed = 0;

    // 1. Warm the lookup table (read-only, respects cache line size)
    size_t table_bytes = eng->table->size * sizeof(eng->table->entries[0]);
    size_t table_warm = (table_bytes < cache_cap) ? table_bytes : cache_cap;
    for (size_t i = 0; i < table_warm; i += CACHE_LINE) {
        __builtin_prefetch((char *)eng->table->entries + i, 0, 3);
    }
    warmed += table_warm;

    // 2. Warm NIC ring buffers (what's left of the cache budget)
    if (warmed < cache_cap) {
        size_t ring_budget = cache_cap - warmed;
        warm_nic_ring(eng->rx_ring, ring_budget);
    }

    // 3. Branchless dry-run through the processing path
```

```
    eng->dry_run = 1;
    struct Packet dummy_pkt = make_dummy_packet();
    handle_packet(eng, &dummy_pkt);  // warms code path + icache
    eng->dry_run = 0;
}
```

Cheat Sheet

TECHNIQUE	WHEN TO USE
`__builtin_prefetch`	You know exactly which addresses you'll need
`_mm_prefetch`	Same, but you want explicit x86 hint control
`volatile` reads	Compiler keeps removing your warming reads
Dry-run mode	Complex code paths, hard to enumerate addresses
Branchless `arr[2]`	Dry-run without branch misprediction cost
Read-only warming	Multi-threaded systems (almost always)
Deep NIC warming	Ultra-low-latency networking
Cache line stride	Always — don't waste cycles on redundant loads
Cache size cap	Always — warming past capacity is worse than useless

Final Thoughts

The biggest lesson I learned: cache warming isn't just about making things faster. Done wrong, it actively makes things slower — blowing out other cores' caches,